AUTOMATED ATM TRANSACTION SIMULATOR

Course: Python for Machine Learning | **Project level:** Professional desktop demonstration

Group of Four: [Member 1, Roll] | [Member 2, Roll] | [Member 3, Roll] | [Member 4, Roll]

Department / College: [Insert] **Guide:** [Insert]

Academic Session: 2026-27 **Submission date:** [Insert]

Important: This is a coursework report template. Fill in your actual details and add authentic output screenshots before submission. The application uses fictional accounts and money.

Certificate (template)

This is to certify that the project titled "Automated ATM Transaction Simulator" has been prepared by the four students named on the cover under the academic project requirements of [Institution]. The undersigned verifier should review and sign the completed report as required by the institution.

Guide name / signature: _____ Date: _____

Acknowledgement

We acknowledge guidance from our course instructor and the resources made available by our department. We thank our group members for contributions to design, programming, testing and documentation.

Abstract

This project demonstrates a desktop ATM using Python, Tkinter and SQLite. It implements demo account authentication, balance inquiry, deposits, withdrawals, two-account money transfers, transaction records, PIN changes, receipt exports and logout. An optional scikit-learn Isolation Forest analyzes recorded transaction amounts to flag statistical outliers for manual review. This is not a real banking or fraud-prevention system.

1. Introduction and Problem Statement

Banking transaction applications combine authentication, data persistence, balance validation and clear user feedback. The exercise is to implement these concepts in a safe local simulation, then introduce simple unsupervised machine learning for anomaly exploration.

2. Objectives

Build a usable GUI; persist fictional transactions; ensure transfers update both accounts together; validate positive amounts; demonstrate salted PIN hashing and login-attempt limits; display history and printable text receipts; compare typical and anomalous transaction amounts in a small ML demonstration.

3. Requirements

Hardware	Software
Desktop/laptop, 4 GB RAM minimum suggested	Python 3.10+, Tkinter, SQLite, VS Code
Local storage for SQLite and receipts	pandas, scikit-learn; matplotlib optional

4. System Architecture

Flow: Tkinter Login Screen → Authentication Service → Dashboard → Operations Service → SQLite Accounts/Transactions → Confirmation/Receipt. The dashboard can also call the Isolation Forest analysis on the logged-in account transaction history.

5. Program Flowchart

Start → Initialize SQLite → Login → {Valid PIN?} → [No: Error/attempt counter] or [Yes: Dashboard] → Select action → Validate request → Atomic database commit or rollback → Display balance/receipt → Dashboard or Logout.

6. Database Design

Table	Key fields	Purpose
accounts	account_no, name, pin_hash, balance_paise, failed_attempts, locked	One row per demo user, integer paise
transactions	id, account_no, kind, amount_paise, note, created_at, reference	Timestamped audit-like demo history

PINs are salted PBKDF2-HMAC-SHA256 digests; values are hashed when accounts are initialized. The schema does not keep plaintext demo PINs. Monetary amounts are parsed using Decimal then stored as integer paise. SQLite BEGIN IMMEDIATE guarantees that both legs of each transfer are committed together.

7. Python Implementation

database.py defines schema and PIN helpers. **atm_operations.py** validates amounts and implements core operations and transaction recording. **main.py** creates a window-based interface with transaction dialogs and Treeview tables. **ml_analysis.py** exports an anomaly analysis CSV. **tests/test_atm.py** verifies important cases.

8. Machine Learning Integration

Algorithm: scikit-learn IsolationForest (unsupervised). Feature: amount_rupees. Prerequisite: at least 8 actual demonstration transactions per account. Parameters: n_estimators=100, contamination=0.15, random_state=42. The model is fit on the available transaction amounts; outputs +1 Typical and -1 Review. Reports are saved as transaction_analysis.csv. An outlier is not equivalent to fraud. A production system would need larger validated datasets, multiple behavioral features, rigorous evaluation and human oversight.

9. Testing and Results

Test case	Expected result
Correct / incorrect PIN	Successful login or error with limit after 3 failures
Deposit and withdrawal	Balance changes by exact decimal amount
Overdraft attempt	Rejected; original balance remains
Unknown recipient	Transfer rejected without partial debit
Valid transfer	Sender debited and recipient credited
PIN change	Old PIN rejected and new PIN accepted
ML on small history	Insufficient-data message until 8 records
History and receipt	Correct reference and amounts displayed

Automated unit tests: five tests supplied covering the core logic. Run **python -m unittest discover -s tests -v** from the project directory; record the actual observed result on your machine.

10. Output Screenshot Placeholders

Insert screenshots of: 01 Login, 02 Dashboard, 03 Deposit, 04 Withdrawal, 05 Transfer, 06 Mini Statement, 07 Full History, 08 PIN Change, 09 ML Analysis, 10 CSV Report, 11 Test Results and 12 Demo Receipt. See screenshots/SCREENSHOT_CHECKLIST.md for the exact capture plan. These are deliberately not fabricated.

11. Group Work Allocation

Role	Deliverable
Student 1	Tkinter interface design, authentication experience
Student 2	SQLite schema, money validation, atomic transfers
Student 3	Isolation Forest anomaly analysis, CSV, limitations
Student 4	Test suite, documentation, demo, packaging

12. Limitations

No real ATM hardware, physical cash, secure remote services, user registration, bank API, PCI-grade controls, real fraud labels or measured classification accuracy. Demo account locks are reset by removing the local demonstration database. The model operates only on transaction amounts and should not automatically block users.

13. Future Scope

Introduce configurable transaction limits, account creation for demos, multi-feature anomaly analysis, a labeled evaluation dataset, richer visual dashboards, report generation and accessibility improvements. Any real deployment would need substantial security design and regulatory review.

14. Conclusion

The simulator combines Python fundamentals, desktop UI, SQLite transactional integrity, input validation and a simple unsupervised ML extension. It illustrates how database reliability and anomaly review can coexist in a learning application without claiming banking-grade security.

15. References

Python 3 documentation: https://docs.python.org/3/

Tkinter documentation: https://docs.python.org/3/library/tkinter.html

SQLite documentation: https://www.sqlite.org/docs.html

scikit-learn IsolationForest documentation: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

pandas documentation: https://pandas.pydata.org/docs/

Submission checklist: Group names filled; app works; tests pass; screenshots are authentic; ZIP includes Python source, requirements, report, README and screenshot files.